Politechnika Wrocławska · Wydział Informatyki i Telekomunikacji

RAPORT – ETAP 1

Badanie wydajności serwisu internetowego ByShutter

Ładowanie zasobów z serwera ze szczególnym uwzględnieniem zdjęć

Temat projektu	Badanie wydajności ładowania zasobów (w szczególności zdjęć) w serwisie portfolio fotograficznego ByShutter, z użyciem narzędzi WebPageTest i Google Lighthouse.
Zespół	Adrian Czubaty, Jan Narożny
Technologie	Frontend: Vue 3, TypeScript, Vite, Tailwind CSS 4. Backend: ASP.NET Core (.NET 10), Entity Framework Core, PostgreSQL 16. Infrastruktura: Docker Compose, serwer Caddy.
Data	21 października 2026

Opis projektu

ByShutter to serwis internetowy typu portfolio fotograficzne, w którym odwiedzający przegląda stronę główną, albumy i pojedyncze zdjęcia, a fotograf zarządza treścią przez panel administracyjny. Aplikacja składa się z klienta działającego w przeglądarce (SPA w Vue 3), interfejsu REST API w ASP.NET Core oraz serwera plików, który wydaje zdjęcia składowane na dysku. Celem projektu jest zmierzenie, jak szybko serwis ładuje zasoby z serwera – zwłaszcza zdjęcia – oraz wskazanie i zweryfikowanie miejsc, w których można ten czas skrócić.

1 Opis projektu i funkcje systemu

Badanym systemem jest serwis **ByShutter** (repozytorium *ByShutter-FrontPage*), powstały jako strona portfolio dla fotografa. Składa się z dwóch aplikacji w jednym repozytorium: klienta `ByShutterPage` i serwera `ByShutterBackend`.

1.1 Funkcje systemu

Część publiczna (odwiedzający):

- strona główna (`/home`) z dużym zdjęciem tytułowym, sekcją „o mnie" i czterema kafelkami portfolio,
- lista albumów (`/albums`) z okładkami oraz widok pojedynczego albumu (`/albums/:id`) – siatka zdjęć,
- podstrony „o mnie" (`/about-me`) i kontakt (`/contact`).

Panel administracyjny (fotograf, po zalogowaniu, ścieżki `/config/*`):

- logowanie (uwierzytelnianie ciasteczkiem sesyjnym),
- przesyłanie zdjęć wraz z metadanymi (nazwa wyświetlana, tekst alternatywny, opis) i zarządzanie nimi,
- tworzenie albumów oraz dodawanie i usuwanie zdjęć z albumów,
- konfiguracja strony głównej: profil strony (zdjęcie tytułowe, zdjęcie profilowe, teksty) i kafelki portfolio.

1.2 Architektura i przepływ danych

Przeglądarka pobiera statyczną aplikację SPA, a następnie dane w formacie JSON z API (`/api/...`). API zwraca metadane (m.in. nazwę pliku zdjęcia), a sam plik graficzny przeglądarka pobiera osobnym żądaniem z serwera plików Caddy, który udostępnia katalog z wgranymi zdjęciami. Metadane trafiają do bazy PostgreSQL, a pliki zdjęć na dysk (wolumen Dockera współdzielony między API a Caddy).

Etap	Żądanie	Odpowiedź
1	Przeglądarka ⟶ serwer: dokument HTML, skrypt JS i style SPA	Pusty szkielet strony, pakiet JS (ok. 337 kB) i CSS (ok. 36 kB)
2	SPA ⟶ API: profil strony głównej	JSON: teksty, identyfikatory zdjęć, kafelki
3	SPA ⟶ API: metadane każdego zdjęcia (osobne żądanie na zdjęcie)	JSON: nazwa pliku, tekst alternatywny
4	Przeglądarka ⟶ Caddy: plik zdjęcia	Binarny plik JPEG/PNG

Tabela 1: Uproszczony przebieg ładowania strony głównej

1.3 Wejście i wyjście systemu

Aktor	Wejście	Wyjście
Odwiedzający	Adres URL, nawigacja po podstronach, przewijanie, kliknięcia w kafelki i albumy	Wyrenderowana strona (HTML/CSS/JS), zdjęcia, teksty, widoki albumów
Administrator	Dane logowania, pliki zdjęć (multipart), metadane zdjęć, dane albumów, kafelków i profilu strony	Potwierdzenia operacji, zaktualizowana zawartość serwisu, ciasteczko sesyjne
Serwer API	Żądania HTTP (JSON, `multipart/form-data`), ciasteczko sesyjne	Odpowiedzi JSON (DTO), kody błędów, rekordy w PostgreSQL, pliki w magazynie zdjęć
Serwer plików (Caddy)	Żądania HTTP(S) o pliki po nazwie	Pliki zdjęć i zasoby statyczne (z katalogu `site`)

Tabela 2: Wejście i wyjście systemu z podziałem na aktorów

W kontekście badania wydajności najważniejsze jest wyjście systemu widziane przez odwiedzającego: **czas, po którym strona jest widoczna i gotowa do użycia**, oraz **ilość danych** (głównie zdjęć), które musi w tym celu przesłać serwer.

1.4 Wstępne obserwacje istotne dla badania

Przegląd kodu i zasobów projektu wskazuje elementy, które mogą wpływać na ładowanie zasobów. Traktujemy je jako hipotezy do zweryfikowania w kolejnych etapach, a nie jako wyniki pomiarów.

- **Duże oryginały zdjęć.** W katalogu zdjęć serwera siedem plików ma od 3,2 do 5,9 MB (łącznie ok. 30 MB). Zdjęcia są wydawane w takiej postaci, w jakiej je wgrano – kod nie generuje pomniejszonych wersji ani nie używa `srcset`.
- **Zdjęcie tytułowe jako tło CSS.** Zdjęcie na górze strony głównej jest ustawiane stylem `background-image`. Do czasu pobrania danych z API używany jest plik zastępczy `ClockImage.png` (ok. 2,1 MB), więc zanim pojawi się właściwe zdjęcie, przeglądarka może pobrać dodatkowy, duży plik.
- **Łańcuch żądań.** Adresy zdjęć są znane dopiero po odpowiedzi API, a metadane każdego zdjęcia pobierane są osobnym żądaniem. Na stronie głównej oznacza to kilka żądań do API przed rozpoczęciem pobierania pierwszego zdjęcia.
- **Czcionki zewnętrzne.** Style importują trzy rodziny czcionek z Google Fonts (`@import` w CSS), co dokłada żądania do zewnętrznych domen.
- **Kompresja i cache.** Konfiguracja Caddy (`file_server`) nie zawiera jawnie dyrektyw kompresji ani nagłówków cache – do sprawdzenia w pomiarach.
- **Leniwe ładowanie.** Atrybut `loading="lazy"` jest używany w widokach albumów, ale nie dla zdjęć kafelków na stronie głównej.

2 Wybór technologii implementacji systemu

Warstwa	Technologia	Uzasadnienie wyboru
Frontend	Vue 3, TypeScript, Vite 8, Vue Router, Pinia	Reaktywny framework komponentowy z niewielkim rozmiarem pakietu; typowanie TypeScript ogranicza błędy; Vite zapewnia szybki build i podział kodu; Router obsługuje podstrony SPA, a Pinia stan aplikacji.
Style	Tailwind CSS 4	Klasy narzędziowe generują tylko używane style, dzięki czemu plik CSS pozostaje mały (ok. 36 kB).
Formularze i HTTP	Axios, VeeValidate, Zod	Klient HTTP z obsługą ciasteczek, walidacja formularzy panelu administracyjnego po stronie klienta z jawnymi schematami.
Backend	ASP.NET Core (.NET 10), minimal API, MediatR, FluentValidation	Wydajna platforma serwerowa; prosty, lekki model endpointów; podział na warstwy (Domain, Application, Infrastructure, Web) i wzorzec CQRS ułatwiają rozwój i testowanie.
Baza danych	PostgreSQL 16, EF Core + Npgsql	Relacyjny model pasuje do powiązań zdjęć, albumów i kafelków; migracje EF Core porządkują zmiany schematu.
Pliki zdjęć i serwer WWW	Caddy 2	Serwer statyczny o prostej konfiguracji, wspierający HTTP/2 i HTTP/3 oraz automatyczne HTTPS.
Uruchamianie	Docker Compose	Powtarzalne środowisko (baza, API i serwer plików), ważne dla porównywalności pomiarów.

Testy	Vitest, Cypress	Testy jednostkowe i end-to-end komponentów oraz przepływów użytkownika.

Tabela 3: Zestawienie technologii użytych w systemie

3 Narzędzia do testów wydajnościowych

Do badania wybraliśmy dwa uzupełniające się narzędzia: **WebPageTest** (pomiar w realnej przeglądarce przez sieć, ze szczegółowym wykresem ładowania) oraz **Google Lighthouse** (audyt laboratoryjny z konkretnymi zaleceniami dotyczącymi zasobów).

3.1 WebPageTest

Narzędzie online (webpagetest.org), które otwiera stronę w prawdziwej przeglądarce z wybranej lokalizacji, na zadanym profilu sieci (np. 4G, Cable) i urządzeniu. Wykonuje wiele powtórzeń testu oraz osobno test pierwszego i kolejnego wejścia na stronę (cache). Dla naszego badania najważniejsze są:

- **wykres kaskadowy (waterfall)** – kolejność, czas i rozmiar każdego żądania, w tym zdjęć i czcionek,
- **rozbicie ruchu według typu zasobu** – liczba żądań i bajtów dla obrazów, JS, CSS, czcionek,
- **filmstrip i wideo ładowania** – moment, w którym pojawia się zdjęcie tytułowe,
- porównanie pierwszego i powtórnego wejścia (wpływ nagłówków cache).

3.2 Google Lighthouse

Narzędzie audytu dostępne w Chrome DevTools (zakładka *Lighthouse*), z linii poleceń i jako PageSpeed Insights. Ładuje stronę w kontrolowanych warunkach (emulacja urządzenia mobilnego i spowolnionej sieci) i zwraca metryki oraz listę audytów. Dla zdjęć szczególnie przydatne są audyty: *Properly size images*, *Efficiently encode images*, *Serve images in next-gen formats*, *Defer offscreen images*, *Preload Largest Contentful Paint image*, *Enable text compression* i *Serve static assets with an efficient cache policy*. Lighthouse wskazuje też element będący LCP, a więc pozwala potwierdzić, czy jest nim zdjęcie tytułowe.

3.3 Porównanie i sposób użycia

Cecha	WebPageTest	Google Lighthouse
Rodzaj pomiaru	Realna przeglądarka, test przez sieć z wybranej lokalizacji	Test laboratoryjny, emulacja urządzenia i sieci
Mocna strona	Szczegółowy waterfall, wiele powtórzeń, test cache	Konkretne rekomendacje i audyty obrazów
Rola w badaniu	Pomiar czasów, liczby żądań i rozmiaru	Diagnoza przyczyn i weryfikacja poprawek

Tabela 4: Rola obu narzędzi w badaniu

Aby wyniki były porównywalne, każdy scenariusz będzie mierzony wielokrotnie (mediana z co najmniej pięciu przebiegów) na tym samym profilu sieci i urządzenia, a strona będzie uruchamiana w jednakowej konfiguracji Docker Compose.

4 Wybrane metryki wydajności

Skupiamy się na metrykach opisujących ładowanie zasobów z serwera, ze szczególnym uwzględnieniem zdjęć. Wybraliśmy pięć metryk czasowych oraz dwie dodatkowe miary ilościowe: rozmiar strony i liczbę żądań. Progi „dobrej” wartości pochodzą z wytycznych Google (web.dev).

4.1 Pięć metryk czasowych

Metryka	Definicja i znaczenie dla serwisu	Próg „dobry”	Narzędzie

TTFB *Time to First Byte*	Czas od rozpoczęcia nawigacji do odebrania pierwszego bajta odpowiedzi serwera. Obejmuje wyszukanie DNS, nawiązanie połączenia, TLS i czas odpowiedzi serwera. Pokazuje, jak szybko serwer zaczyna wydawać zasoby.	≤ 0,8 s	WebPageTest, Lighthouse (*server response time*)
FCP *First Contentful Paint*	Czas do wyświetlenia pierwszego fragmentu treści (tekst, obraz, tło). Pokazuje, kiedy użytkownik widzi, że strona „ruszyła"; zależy od pobrania HTML, CSS, JS i czcionek.	≤ 1,8 s	WebPageTest, Lighthouse
LCP *Largest Contentful Paint*	Czas wyrenderowania największego elementu widocznego w oknie – w ByShutter najpewniej zdjęcia tytułowego. Najbardziej bezpośrednia miara wpływu rozmiaru i kolejności ładowania zdjęć na odbiór strony.	≤ 2,5 s	WebPageTest, Lighthouse
Load Time *czas ładowania*	Czas do zakończenia zdarzenia `load`, gdy pobrane są wszystkie zasoby wskazane w dokumencie (w WebPageTest: *Document Complete* oraz *Fully Loaded* – po ustaniu ruchu sieciowego). Wskazuje, ile trwa pobranie wszystkich zasobów, w tym całej siatki zdjęć.	Brak formalnego progu; zależy od strony	WebPageTest
CLS *Cumulative Layout Shift*	Suma nieoczekiwanych przesunięć układu strony podczas ładowania. Obrazy bez zadeklarowanych wymiarów powodują przesuwanie treści po ich pobraniu, więc metryka mierzy jakość ładowania zdjęć w układzie strony.	≤ 0,1	WebPageTest, Lighthouse

Tabela 5: Pięć wybranych metryk czasowych i jakościowych

4.2 Metryki dodatkowe: rozmiar strony i liczba żądań

Metryka	Definicja i uzasadnienie	Narzędzie
Rozmiar strony *page weight*	Łączna ilość danych przesłanych z serwera podczas ładowania (transfer, w kB/MB), raportowana także osobno dla typów zasobów: obrazy, JS, CSS, czcionki. Zdjęcia są najcięższym zasobem serwisu, dlatego udział obrazów w całości transferu jest kluczową miarą badania. Porównamy też rozmiar po kompresji (transfer) z rozmiarem po dekompresji.	WebPageTest (*Content breakdown*), Lighthouse (*Total byte weight*)
Liczba żądań *request number*	Liczba żądań HTTP wykonanych do pełnego załadowania strony, z podziałem na typ zasobu i domenę (własny serwer, API, Google Fonts). Każde żądanie dokłada koszt opóźnienia sieci; w serwisie dochodzą żądania do API, osobne żądania o metadane każdego zdjęcia i żądania o same pliki graficzne.	WebPageTest (*Requests*), Lighthouse (*Network requests*)

Tabela 6: Metryki ilościowe

4.3 Powiązanie metryk z badanymi zasobami

Wybrane metryki pokrywają kolejne fazy ładowania: odpowiedź serwera (TTFB), pierwsze wyświetlenie (FCP), pojawienie się głównego zdjęcia (LCP), pełne pobranie zasobów (Load Time) i stabilność układu przy

doczytywaniu obrazów (CLS). Rozmiar strony i liczba żądań wyjaśniają, *dlaczego* metryki czasowe mają określone wartości. W następnych etapach zmierzymy stan wyjściowy serwisu, a następnie sprawdzimy wpływ zmian dotyczących zdjęć (zmniejszenie i konwersja formatu, `srcset`, `loading="lazy"`, preload zdjęcia tytułowego) oraz serwera (kompresja, nagłówki cache).

5 Źródła

- web.dev – Time to First Byte, First Contentful Paint, Largest Contentful Paint, Cumulative Layout Shift: web.dev/articles/ttfb, fcp, lcp, cls.
- Dokumentacja Lighthouse: developer.chrome.com/docs/lighthouse.
- Dokumentacja WebPageTest: docs.webpagetest.org.
- Repozytorium projektu: ByShutter-FrontPage (kod klienta `ByShutterPage` i serwera `ByShutterBackend`).